06 - Clustering Algorithms

Alessandro Bregoli

2025

Introduction

In this tutorial we will discuss the following topics:

Introduction

In this tutorial we will discuss the following topics:

- Prototype Based Clustering Algorithms
-
-
-
-
-

Introduction

In this tutorial we will discuss the following topics:

- Prototype Based Clustering Algorithms
- Hierarchical Clustering Algorithms

Introduction

In this tutorial we will discuss the following topics:

- Prototype Based Clustering Algorithms
- Hierarchical Clustering Algorithms
- Density Based Clustering Algorithm

Introduction

In this tutorial we will discuss the following topics:

- Prototype Based Clustering Algorithms
- Hierarchical Clustering Algorithms
- Density Based Clustering Algorithm

Furthermore we will cover the following knime related topics:

-
-
-

Introduction

In this tutorial we will discuss the following topics:

- Prototype Based Clustering Algorithms
- Hierarchical Clustering Algorithms
- Density Based Clustering Algorithm

Furthermore we will cover the following knime related topics:

- Use Scatter Plot/ Scatter Plot Matrix
-
-

Introduction

In this tutorial we will discuss the following topics:

- Prototype Based Clustering Algorithms
- Hierarchical Clustering Algorithms
- Density Based Clustering Algorithm

Furthermore we will cover the following knime related topics:

- Use Scatter Plot/ Scatter Plot Matrix
- Use the Color Manager node to color the output of a Scatter Plot
-

Introduction

In this tutorial we will discuss the following topics:

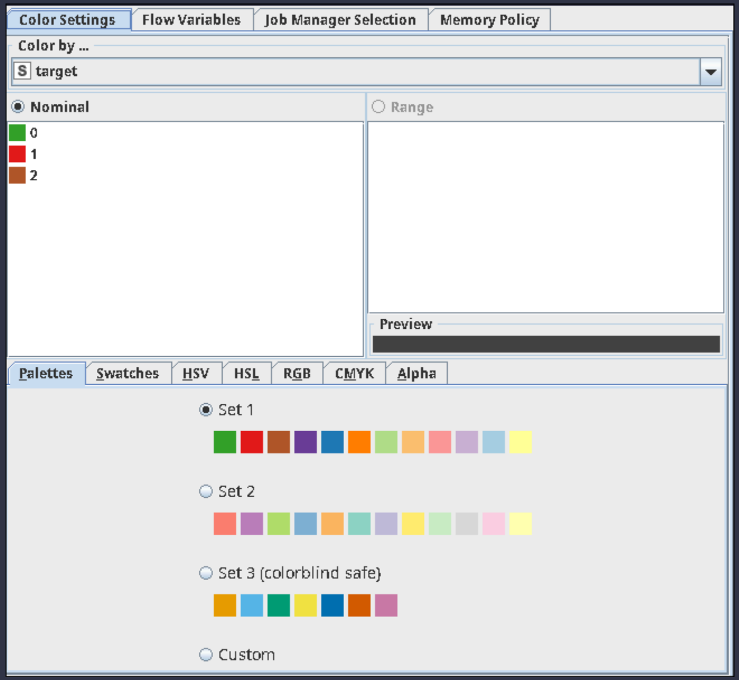
- Prototype Based Clustering Algorithms
- Hierarchical Clustering Algorithms
- Density Based Clustering Algorithm

Furthermore we will cover the following knime related topics:

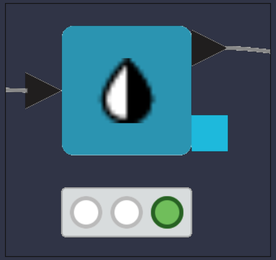
- Use Scatter Plot/ Scatter Plot Matrix
- Use the Color Manager node to color the output of a Scatter Plot
- Load a dataset using python

Color Manager/Scatter Plot Matrix

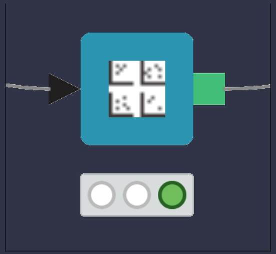
Knime allows to associate colors to each row using the *Color Manager* node:



Color Manager

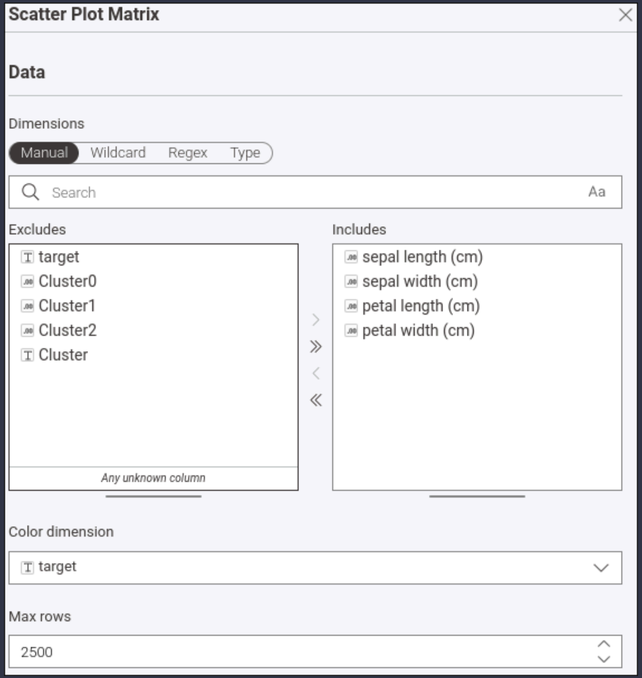


Scatter Plot Matrix

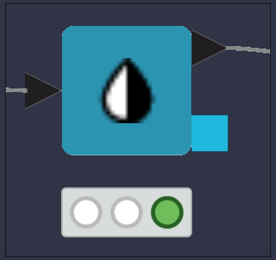


Color Manager/Scatter Plot Matrix

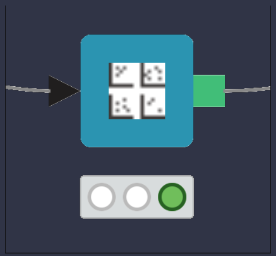
The colors can be used with the *Scatter Plot Matrix* node:



Color Manager

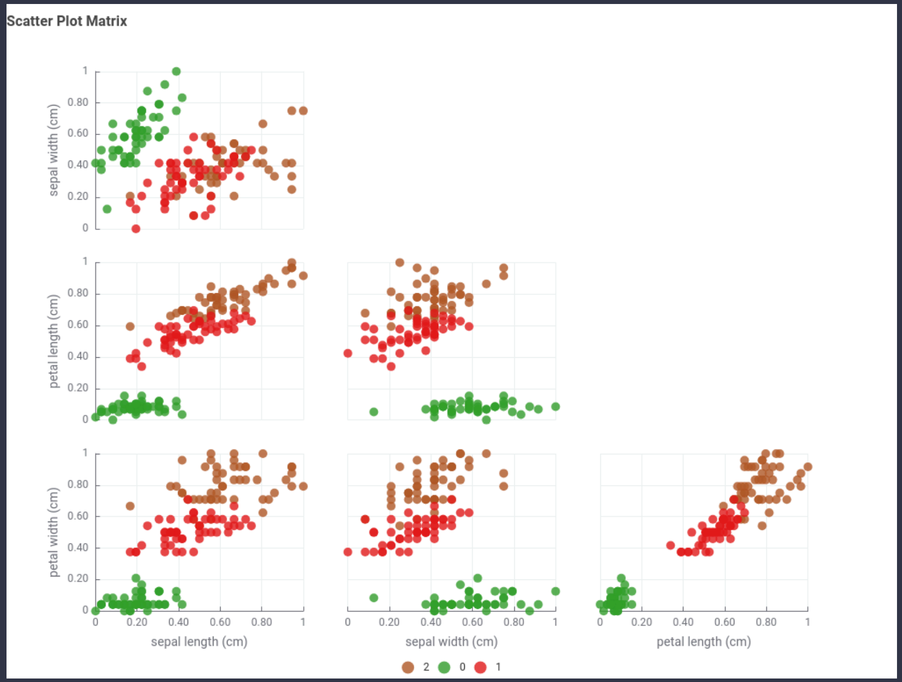


Scatter Plot Matrix



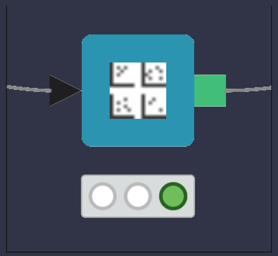
Color Manager/Scatter Plot Matrix

The result is a scatterplot matrix where each point is colored according to the color manager settings:



Color Manager

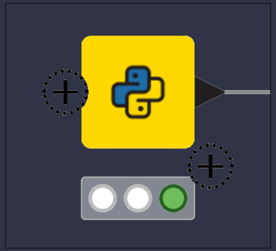
Scatter Plot Matrix



Iris Dataset

For this tutorial we will use the **Iris Dataset**.

Python Script

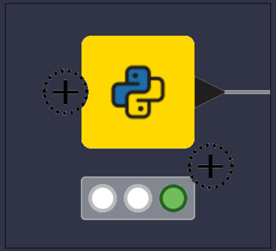


Iris Dataset

For this tutorial we will use the **Iris Dataset**.

To import his dataset we will use the *python script* node.

Python Script

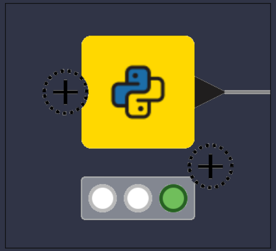


Iris Dataset

For this tutorial we will use the **Iris Dataset**.

To import his dataset we will use the *python script* node.

Python Script



Python Script

Python Script

Flow variables knio.flow_va…
knime.workspace STRING
Output Table 1 knio.output_…

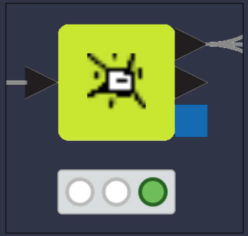
```
import knime.scripting.io as knio

# This example script creates an output table containing randomly drawn integers using numpy and pandas.

import numpy as np
import pandas as pd
from sklearn.datasets import load_iris

iris_dataset = load_iris()
df = pd.DataFrame(iris_dataset.data, columns=iris_dataset.feature_names)
df["target"] = iris_dataset.target.astype(str)

knio.output_tables[0] = knio.Table.from_pandas(df)

```

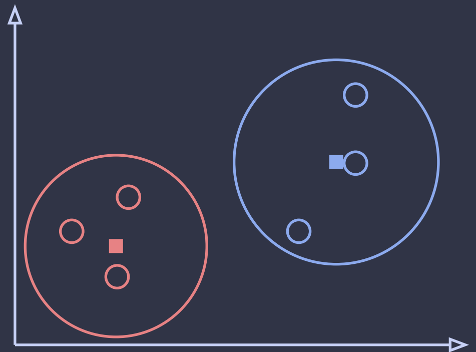
Prototype Based

K-Means

The **K-means** defines a **prototype** in terms of **centroid**.

K-Means



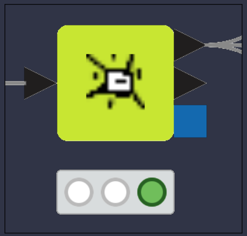


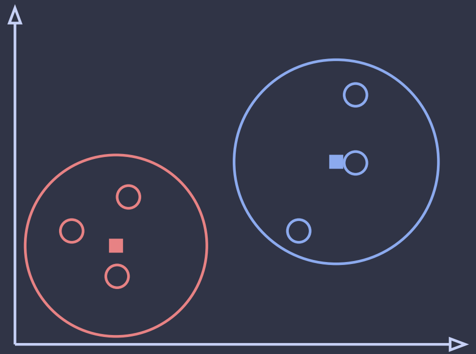
K-Means

The **K-means** defines a **prototype** in terms of **centroid**.

The **main** parameter for **K-Means** is the **number of clusters/ centrodis**.

K-Means





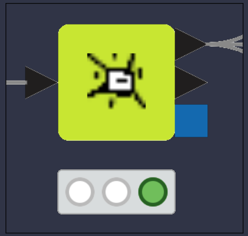
K-Means

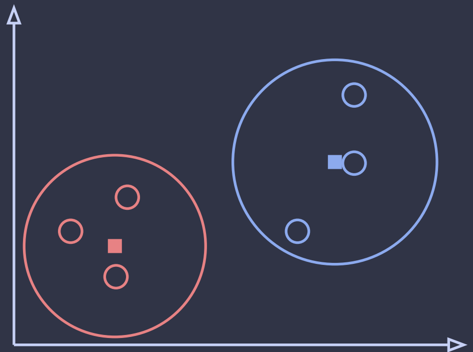
The **K-means** defines a **prototype** in terms of **centroid**.

The **main** parameter for **K-Means** is the **number of clusters/ centrodis**.

Usually, in K-means, the centroid is the **mean** of the **attributes** in a specific **group** of **objects**.

K-Means





K-Means

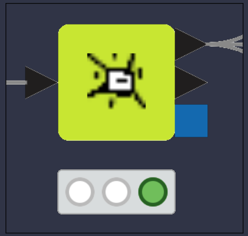
The **K-means** defines a **prototype** in terms of **centroid**.

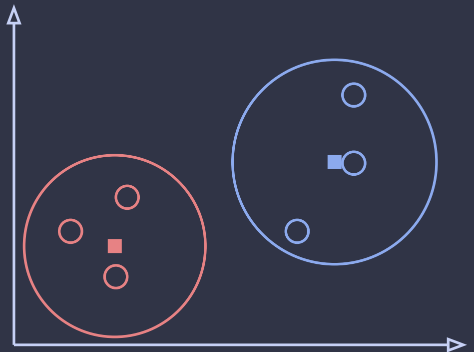
The **main** parameter for **K-Means** is the **number of clusters/ centrodis**.

Usually, in K-means, the centroid is the **mean** of the **attributes** in a specific **group** of **objects**.

The **distance** between the **centroid** and each **element** is usually computed with the **euclidean distance**.

K-Means





K-Means

The **K-means** defines a **prototype** in terms of **centroid**.

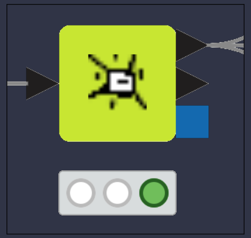
The **main** parameter for **K-Means** is the **number of clusters/ centrodis**.

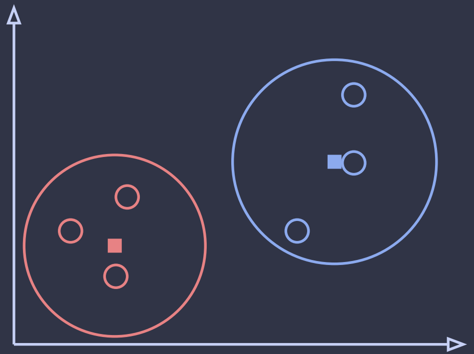
Usually, in K-means, the centroid is the **mean** of the **attributes** in a specific **group** of **objects**.

The **distance** between the **centroid** and each **element** is usually computed with the **euclidean distance**.

Since the **Euclidean distance** is used, it is **necessary** to **normalize** the **features** before applying this clustering algorithm.

K-Means

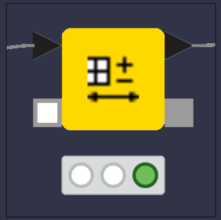




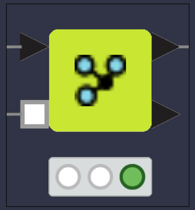
K-Menoids

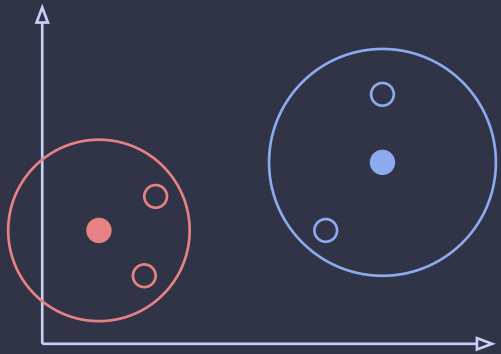
The **K-Menoids** is similar to the K-Means algorithm.

Distance Matrix Calculator



K-Menoids



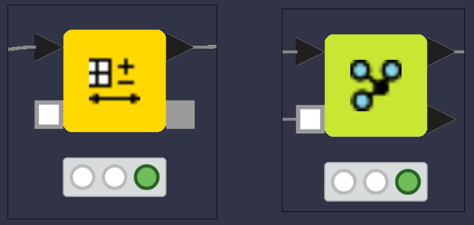


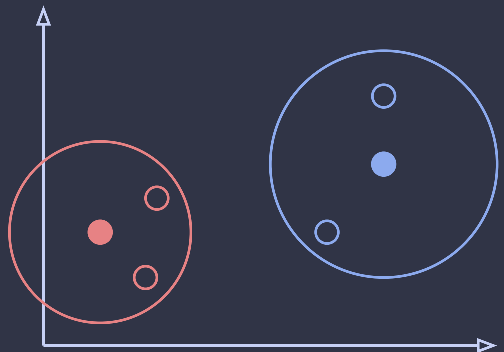
K-Menoids

The **K-Menoids** is similar to the K-Means algorithm.

The **main difference** is that, for K-Menoids, the **prototype** is an **actual data point**.

Distance Matrix Calculator **K-Menoids**

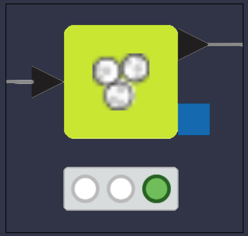


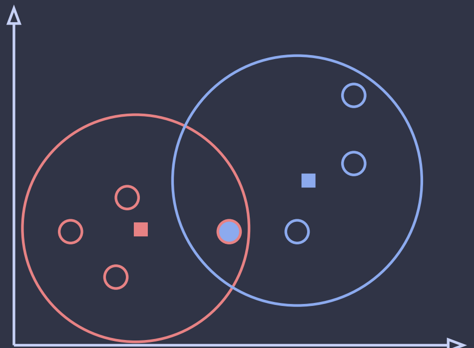


Fuzzy c-Means

The **Fuzzy c-Means** is an extension of the K-Means where each **element** is assigned with a certain **weight** to each **cluster**.

Fuzzy c-Means



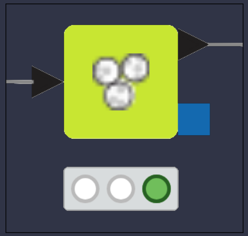


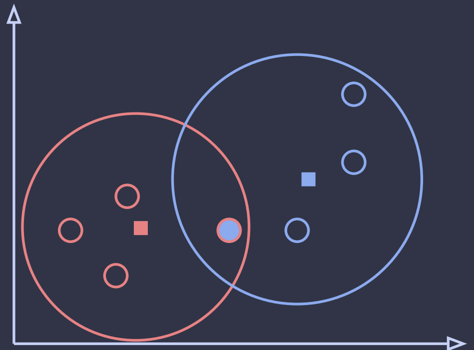
Fuzzy c-Means

The **Fuzzy c-Means** is an extension of the K-Means where each **element** is assigned with a certain **weight** to each **cluster**.

Fuzzy c-Means yields a k-means-identical result when each **element** is **hard-assigned** to its closest cluster center.

Fuzzy c-Means

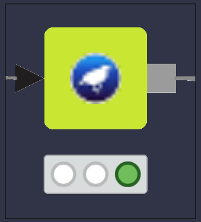




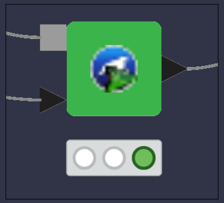
Gaussian Mixture Model

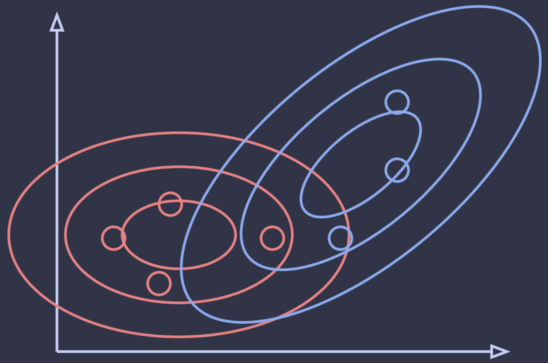
Gaussian Mixture Models (GMM) describes the dataset with a mixture of **Multivariate Gaussian distributions**.

EM



Weka Cluster Assigner



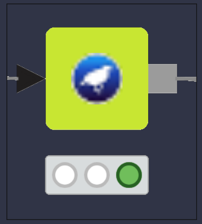


Gaussian Mixture Model

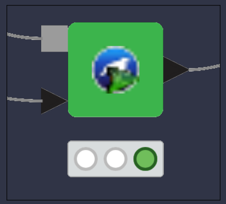
Gaussian Mixture Models (GMM) describes the dataset with a mixture of **Multivariate Gaussian distributions**.

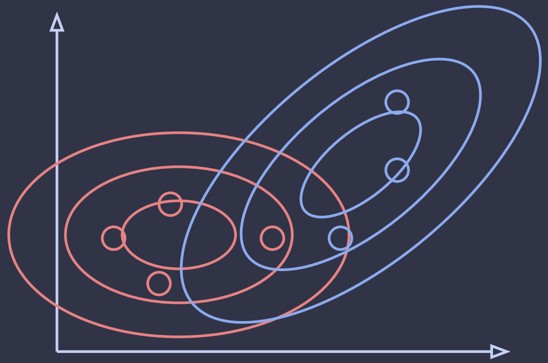
Each **component/cluster** is modeled with a **Multivariate Gaussian Distribution**, thus for each component the GMM learn a **mean vector** and a **covariance matrix**.

EM



Weka Cluster Assigner



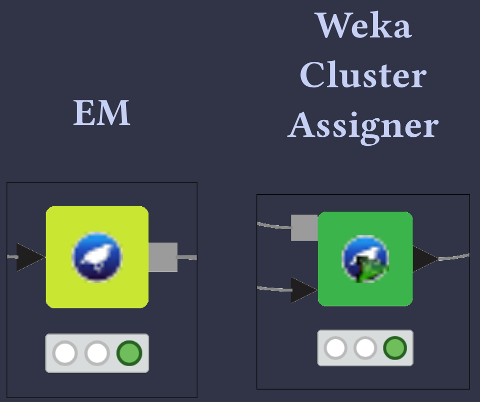


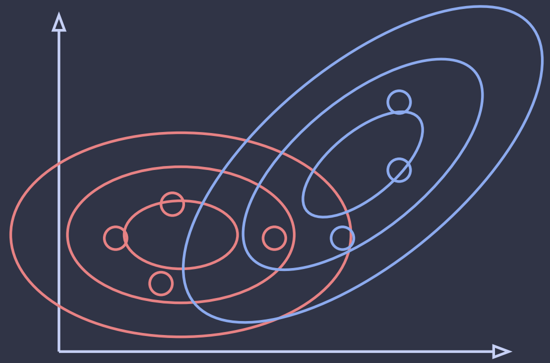
Gaussian Mixture Model

Gaussian Mixture Models (GMM) describes the dataset with a mixture of **Multivariate Gaussian distributions**.

Each **component/cluster** is modeled with a **Multivariate Gaussian Distribution**, thus for each component the GMM learn a **mean vector** and a **covariance matrix**.

Each element is assigned a **probability** of **belonging** to **every cluster** by the GMM.





Gaussian Mixture Model

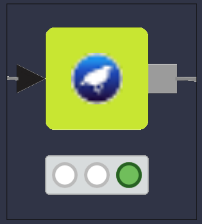
Gaussian Mixture Models (GMM) describes the dataset with a mixture of **Multivariate Gaussian distributions**.

Each **component/cluster** is modeled with a **Multivariate Gaussian Distribution**, thus for each component the GMM learn a **mean vector** and a **covariance matrix**.

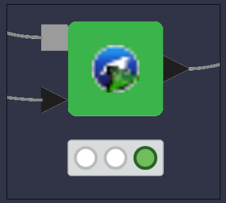
Each element is assigned a **probability** of **belonging** to **every cluster** by the GMM.

Since the **clusters** are **not spherical**, GMM is **less sensitive** to **scale**.

EM



Weka Cluster Assigner



Hands on (minutes)

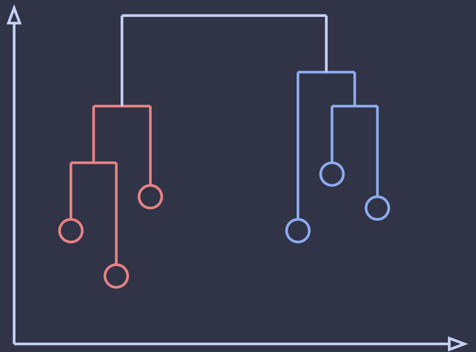
- **Load** the **iris dataset** using the python script node
- **Shuffle** the rows with the *shuffle* node.
- **Normalize** the features
- **Test** the following cluster algorithms:
  - k-means
  - Fuzzy k-means
  - k-menoids
    - *hint: use the k-menoids node in combination with the distance matrix calculate node*
  - GMM
    - *hint: use the EM weka node*
- Show the results of the clustering using the *scatter plot matrix* node.
- Compare the clustering against the target variable using a set of *pie-plots*.

Hierarchical Agglomerative

Hierarchical Clustering

Agglomerative Hierarchical Clustering starts with all elements as individual clusters. Then, iteratively, it merges the **closest pair** of **clusters**.

Hierarchical Clustering

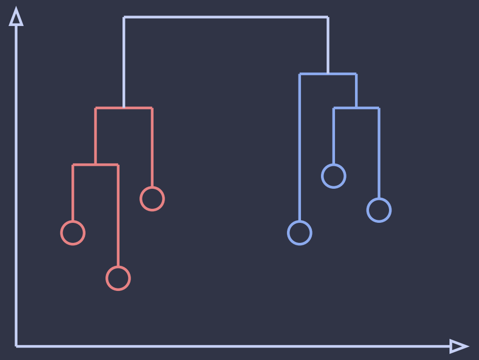


Hierarchical Clustering

Agglomerative Hierarchical Clustering starts with all elements as individual clusters. Then, iteratively, it merges the **closest pair** of **clusters**.

This algorithm require to select a distance measure such as the **Euclidean distance** or the **Manhattan Distance**.

Hierarchical Clustering



Hierarchical Clustering

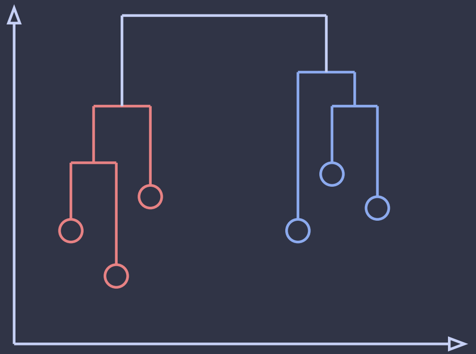
Agglomerative Hierarchical Clustering starts with all elements as individual clusters. Then, iteratively, it merges the **closest pair** of **clusters**.

This algorithm require to select a distance measure such as the **Euclidean distance** or the **Manhattan Distance**.

Furthermore, the Agglomerative Hierarchical clustering require to select a strategy for computing the distance between two clusters:

- **Min/Single Linkage**: distance between the closets elements of the two clusters.
- **Max/Complete Linkage**: distance between the farthest elements of the two clusters.
- **Average Linkage**: average distance between all the possible pairs of elements between the two clusters.

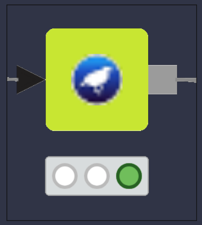
Hierarchical Clustering



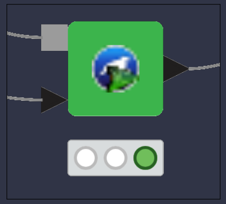
Hierarchical Clustering

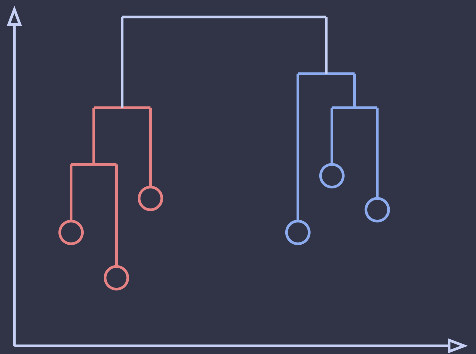
The *Hierarchical Cluster* provided by weka provided more strategies for computing the distance between clusters such as the **ward** strategy.

Hierarchical Clusterer



Weka Cluster Assigner



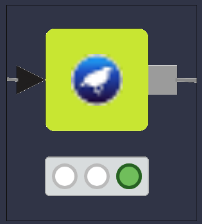


Hierarchical Clustering

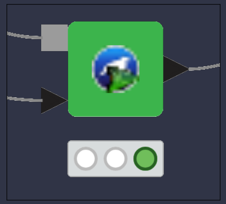
The *Hierarchical Cluster* provided by weka provided more strategies for computing the distance between clusters such as the **ward** strategy.

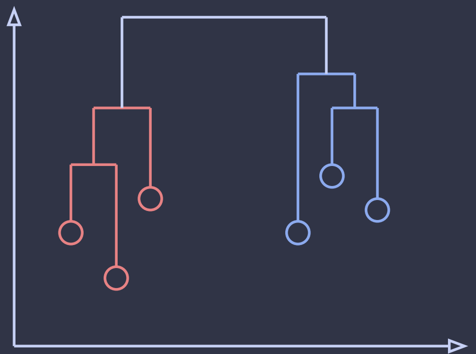
Ward's method merges the two clusters causing the smallest increase in total within-cluster variance to produce compact, spherical clusters.

Hierarchical Clusterer



Weka Cluster Assigner





Hands on (20 minutes)

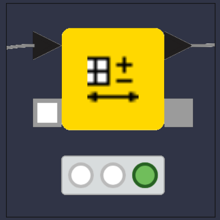
- **Extend** the previous workflow using the Hierarchical Clustering using the Euclid distance and the following strategies:
  - Single Linkage
  - Complete Linkage
  - Average Linkage
  - Ward
- **Evaluate** each cluster using:
  - Scatter plot matrix
  - Pie chart

Density Based

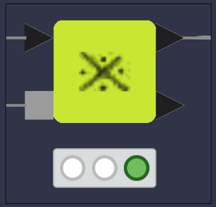
DBSCAN

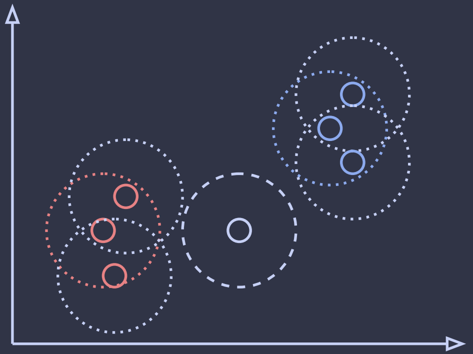
Density based algorithms, such as **DBSCAN**, aim to find **dense region of objects** separated by sparse/empty regions.

Distance Matrix Calculator



DBSCAN



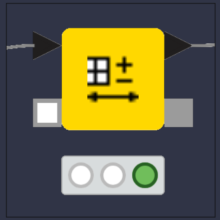


DBSCAN

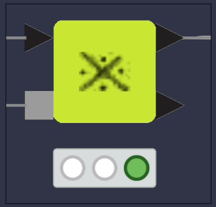
Density based algorithms, such as **DBSCAN**, aim to find **dense region of objects** separated by sparse/empty regions.

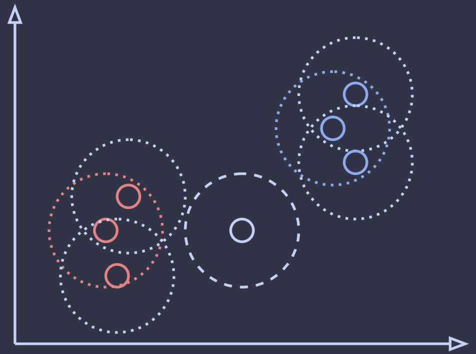
The *DBSCAN* node allows to implement such approach exploiting any distance metric provided by the node *Distance Matrix Calculator*.

Distance Matrix Calculator



DBSCAN





DBSCAN

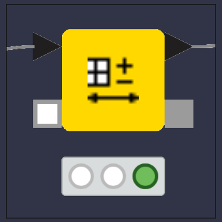
Density based algorithms, such as **DBSCAN**, aim to find **dense region of objects** separated by sparse/empty regions.

The *DBSCAN* node allows to implement such approach exploiting any distance metric provided by the node *Distance Matrix Calculator.*

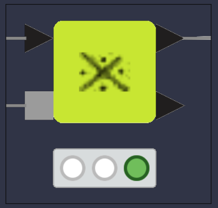
DBSCAN identify 3 kind of points:

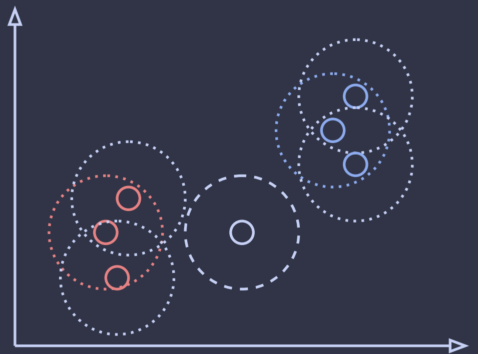
- **Core points**: points closely surrounded by many other points.
- **Border points**: points close to core points.
- **Noise points**: points far from any other point

Distance Matrix Calculator



DBSCAN





DBSCAN

Density based algorithms, such as **DBSCAN**, aim to find **dense region of objects** separated by sparse/empty regions.

The *DBSCAN* node allows to implement such approach exploiting any distance metric provided by the node *Distance Matrix Calculator.*

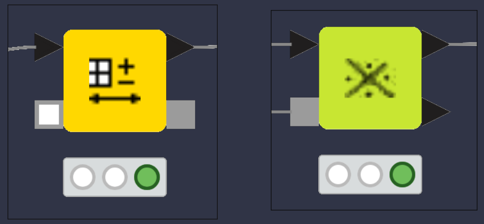
DBSCAN identify 3 kind of points:

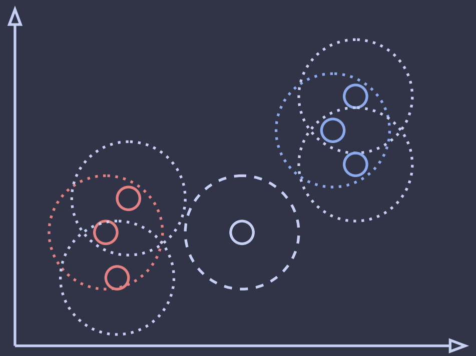
- **Core points**: points closely surrounded by many other points.
- **Border points**: points close to core points.
- **Noise points**: points far from any other point

The identification of such nodes is based on two parameters:

- **Epsilon**: maximum distance for considering two nodes close
- **Minimim Points**: minimum number of close points required to be considered a **core point**.

Distance Matrix Calculator **DBSCAN**





Hands on (15 minutes)

- **Extend** the previous workflow
- **Compute** the **Euclidean distance**
- **Cluster** the dataset using DBSCAN
- **Try** different values of **Epsilon** and **Minimum Points**.
- **Evaluated** the clusters obtained using:
  - Scatter plot matrix
  - Pie Chart